

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	NARAPPAGARI SRAVYA	Reg. No.:	192472192
Section / Batch:	BE CSE AI / 2024	Date:	13-08-2026

Code of Conduct

I NARAPPAGARI SRAVYA(192472192)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SRAVYA

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.

Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.

Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.

Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.

Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

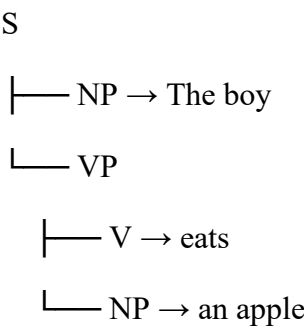
1. CFG Trees vs Dependency Parsing

CFG Trees

A **Context-Free Grammar (CFG)** represents a sentence using hierarchical constituents such as noun phrases (NP) and verb phrases (VP).

Example:

The boy eats an apple.

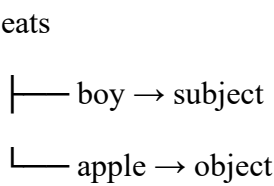


CFG is useful for understanding the **phrase structure** of a sentence.

Dependency Parsing

Dependency parsing represents relationships directly between words. One word acts as the **head**, and other words depend on it.

For the same sentence:



Comparison

Feature	CFG	Dependency Parsing
Main representation	Phrase structure	Word-to-word relationships
Focus	Constituents	Dependencies
Subject/object relations	Indirect	Direct
Long-distance relationships	Can be complex	Easier to represent
Useful for	Grammar and syntax	Semantic relationships and information extraction

Justification

Dependency parsing is more effective for capturing relationships between words because it directly identifies relations such as subject, object, modifier, and dependency.

However, CFG is still valuable when the main requirement is **constituent-based grammatical analysis**.

2. Top-Down Parsing vs Earley Parsing

Top-Down Parsing

Top-down parsing starts with the start symbol, usually S, and attempts to generate the input sentence.

$S \rightarrow NP VP$

It then expands the grammar rules until they match the input.

Advantages:

- Simple to understand.
- Easy to implement for simple grammars.
- Works well for predictable input.

Limitations:

- May require backtracking.
- Can repeat parsing work.
- Less effective with ambiguity.
- Difficult to handle incomplete input efficiently.

Earley Parsing

Earley parsing uses a dynamic programming approach and maintains partially completed parsing states.

It has three main operations:

1. **Prediction**
2. **Scanning**
3. **Completion**

It can handle **ambiguous, incomplete, and complex sentences**.

Comparison

Feature	Top-Down	Earley
Approach	Starts from grammar root	Maintains parsing states
Backtracking	Often required	Reduced
Ambiguous input	Less efficient	Handles well
Incomplete input	Poorer	Better
General CFG support	Limited depending on implementation	Yes
Dynamic applications	Less suitable	More suitable

Justification

Earley parsing is more suitable for real-time dynamic input when users may provide incomplete or ambiguous sentences. It avoids unnecessary repeated parsing and can maintain multiple possible interpretations.

3. CFG vs PCFG vs Neural Parsing

Sentence:

“She saw the man with a telescope.”

This sentence is ambiguous.

Interpretation 1

She used a telescope to see the man.

She → saw → man

↘ telescope

Interpretation 2

The man had the telescope.

She → saw → man

↘ telescope

CFG

CFG can generate **both possible parse trees**, but basic CFG does not know which interpretation is more likely.

Advantage: Simple and interpretable.

Limitation: Cannot naturally rank competing interpretations.

PCFG

PCFG assigns probabilities to grammar rules.

For example:

$P(\text{Parse 1}) = 0.75$

$P(\text{Parse 2}) = 0.25$

The system can select the parse with the higher probability.

Advantage: Better ambiguity resolution than basic CFG.

Limitation: Depends on the quality of the training data and grammar probabilities.

Neural Parsing

Neural parsers use machine-learning models to learn syntactic patterns from large datasets.

They can use:

- Word context
- Sentence context
- Learned representations
- Linguistic patterns

They generally perform better on complex real-world language than manually designed grammars.

Comparison

Feature	CFG	PCFG	Neural Parsing
Handles ambiguity	Generates alternatives	Ranks alternatives	Learns likely interpretation
Context awareness	Low	Moderate	High
Training data	Not necessarily required	Required for probabilities	Usually large datasets
Interpretability	High	High	Lower
Real-world performance	Limited	Better	Generally strongest

Justification

For **real-world NLP applications**, **neural parsing is generally the most effective** because it can learn complex contextual patterns and resolve ambiguity more accurately.

However, **PCFG is useful when interpretability, simplicity, and computational efficiency are important.**

4. Feature Structures vs Subcategorization Frames

These two methods solve different but related problems.

Feature Structures

Feature structures represent grammatical properties of words.

Example:

Subject:

Number = Singular

Person = 3

Verb:

Number = Singular

Person = 3

For:

She runs.

The features match:

She → Singular, 3rd person

runs → Singular, 3rd person

Therefore, the sentence satisfies subject–verb agreement.

For:

She run.

The features do not match.

Subcategorization Frames

Subcategorization frames describe **what types of arguments a verb requires**.

Examples:

eat → Subject + Object

give → Subject + Object + Recipient

sleep → Subject

For:

The boy gave the teacher a book.

The verb **gave** expects:

- Subject → boy
- Recipient → teacher
- Object → book

Comparison

Feature	Feature Structures	Subcategorization Frames
Main purpose	Grammatical features	Verb argument requirements
Subject–verb agreement	Excellent	Limited
Number/person	Yes	No
Verb arguments	Limited	Excellent
Grammatical correctness	Strong	Supports structural correctness

Justification

Feature structures provide better support for enforcing grammatical correctness, especially subject–verb agreement.

However, subcategorization frames are essential for ensuring that verbs receive the **correct number and type of arguments**.

Therefore, the best NLP grammar system should use **both together**.

5. Transition-Based vs Graph-Based Dependency Parsing

Transition-Based Parsing

Transition-based parsing builds the dependency tree **step by step** using parsing actions.

Typical actions include:

- **Shift**
- **Reduce**
- **Left-Arc**
- **Right-Arc**

A classifier or neural model decides which action should be performed next.

Advantages

- Very fast.
- Low computational cost.
- Suitable for real-time applications.
- Works efficiently on large datasets.

Limitations

- Early mistakes can affect later decisions.
- Greedy decisions can reduce accuracy.
- May have difficulty with some long-distance dependencies.

Graph-Based Parsing

Graph-based parsing considers possible dependency relationships and searches for the **best overall dependency tree**.

Instead of making only one local decision, it evaluates relationships globally.

Advantages

- Usually more globally consistent.
- Better at handling complex dependency structures.
- Less affected by individual local decisions.

Limitations

- Computationally more expensive.
- Requires more processing for large datasets.
- Generally slower than transition-based approaches.

Comparison

Feature	Transition-Based	Graph-Based
Decision-making	Local, step-by-step	Global
Speed	Very fast	Slower
Memory/computation	Lower	Higher
Error propagation	Higher	Lower
Complex dependencies	Moderate	Strong

Real-time use	Excellent	Less suitable
Large-scale processing	Excellent	More computationally demanding

Justification

For a **large-scale NLP system requiring fast dependency parsing**, **transition-based parsing is generally more suitable** because it provides high speed and lower computational cost.

If the application prioritizes **maximum global parsing accuracy over speed**, graph-based parsing may be preferred.

Overall Conclusion

- **CFG** → Best for phrase/constituent structure.
- **Dependency Parsing** → Best for direct word relationships.
- **Earley Parsing** → Better for ambiguous and incomplete dynamic input.
- **Neural Parsing** → Generally best for complex real-world ambiguity.
- **Feature Structures** → Best for grammatical agreement.
- **Subcategorization Frames** → Best for verb argument structures.
- **Transition-Based Parsing** → Best when speed and scalability are priorities.